

4 - Intro to Pandas

December 12, 2025

1 Part III. Data Manipulation with Pandas

1.1 Chapter 13.

Pandas is a package built on top of NumPy that offers a convenient interface for manipulating complex datasets. Pandas implements operations familiar to database frameworks and spreadsheet programs.

The essential data structure implemented by Pandas is the **DataFrame**. A data frame is a collection of multidimensional data points that can be thought of as a table. Each column of a data frame is implemented as a **Series**.

A common alias for pandas in Python programs is `pd`.

```
[ ]: ## Installing pandas  
  
%pip install pandas --quiet
```

```
[ ]: import numpy as np  
import pandas as pd # Import pandas as give it the alias `pd`
```

1.1.1 The Pandas Series object

A Pandas **Series** is a one-dimensional array of indexed data. It can be created from a list or array:

```
[ ]: data = pd.Series([0.25, 0.5, 0.75, 1.0])  
print(data)
```

A **Series** is a sequence of values with an explicit sequence of *indices*, accessed through attributes `values` and `index`.

```
[ ]: print(data.values)
```

```
[ ]: print(data.index)
```

1.1.2 The Pandas DataFrame object

The **Series** object is a building block of the Pandas **DataFrame**. We will now create a Pandas **DataFrame**, starting with the creation of a **Series**:

```
[ ]: ## Creating a Series from a dictionary
## State names become indices, area become values

area_dict = {'California': 423967, 'Texas': 695662, 'Florida': 170312,
             'New York': 141297, 'Pennsylvania': 119280}
area = pd.Series(area_dict)
print(area)
```

```
[ ]: ## Creating a Series from a dictionary
## State names become indices, population become values

population_dict = {'California': 39538223, 'Texas': 29145505,
                  'Florida': 21538187, 'New York': 20201249,
                  'Pennsylvania': 13002700}
population = pd.Series(population_dict)
population
```

```
[ ]: ## Create a DataFrame using population and area

states = pd.DataFrame({"population": population,
                       "area": area})

print(states)
```

1.2 Chapter 14. Data Indexing and Selection

1.2.1 Series as Dictionary

Series provides a mapping from a collection of *keys* to a collection of *values*:

```
[ ]: import pandas as pd
data = pd.Series([0.25, 0.5, 0.75, 1.0],
                 index=['a', 'b', 'c', 'd'])
data
```

```
[ ]: print(data['b'])
```

Checking membership with in:

```
[ ]: 'a' in data
```

1.2.2 Series as one-dimensional array

```
[ ]: # slicing by explicit key-index

data['a': 'c']
```

```
[ ]: # slicing by implicit index number

data[0:2]
```

```
[ ]: s = pd.Series([0.1 ,0.2 ,0.3, 0.4, 0.5], index=[2,3,4,5,6])

print(s[0:4])
```

```
[ ]: # masking

data[(data > 0.3) & (data < 0.8)]
```

1.2.3 Indexers: loc and iloc

If your **Series** has explicit integer index, then you can perform the indexing operation with `data[i]` where `i` is the explicit index.

However, if your indices are integers, there can be ambiguity when performing indexing. To prevent confusion, Pandas provides special indexer functions that explicitly address the kind of index you want to use.

The `loc` attribute performs indexing using the explicit index.

```
[ ]: data = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])
```

```
[ ]: data.loc[1:3]
```

The `iloc` attribute performs implicit indexing.

```
[ ]: data.iloc[1:3]
```

1.2.4 Data selection in DataFrames

We can think of a **DataFrame** as a *dictionary* of related **Series** objects (columns):

```
[ ]: area = pd.Series({'California': 423967, 'Texas': 695662,
                    'Florida': 170312, 'New York': 141297,
                    'Pennsylvania': 119280})
pop = pd.Series({'California': 39538223, 'Texas': 29145505,
                'Florida': 21538187, 'New York': 20201249,
                'Pennsylvania': 13002700})
data = pd.DataFrame({'area':area, 'pop':pop})
data
```

In the above example, we can access the individual **Series** that make up the columns of the **DataFrame** using dictionary-style indexing:

```
[ ]: data['area']
```

We can also use the dictionary-style indexing to modify the **DataFrame**, creating a new column:

```
[ ]: data['density'] = data['pop'] / data['area']

print(data)
```

1.2.5 Data filtering

We can use indexing to filter subsets of a `DataFrame`. For instance, assume you would like to select the states with a population density between 100 and 130:

```
[ ]: ## Filtering the DataFrame to include states with population density between
      ↪ 100 and 130:

d_sub = data[(100 < data['density']) & (data['density'] < 130)] # Using the
      ↪ `bitwise` operator &

print(d_sub)
```

1.3 Chapter 16. Handling missing data

In the real world, data is rarely clean and homogeneous. It is not uncommon for real datasets to have missing data. To make matters more complicated, different data sources may use different approaches to indicate missing data.

In Pandas, missing data is often referred to as `NaN` (not a number), `NA`, or *null*.

1.3.1 Missing data in Pandas

By default, Pandas handles missing data using a *sentinel-based* approach. With sentinel values `NaN` or `None` being used to represent missing data.

NaN or None? The `NaN` type can only be assigned to `float` in Pandas. Whereas `None` can be used with any type, including structured ones.

The downside of `None` is that operations are done at the Python level, which are slower. It also does not have arithmetic operations associated with it.

When handling numerical data, it is recommended to use `np.NaN` to indicate missing data:

```
[ ]: vals1 = np.array([1, None, 2, 3])
      print(vals1)

      print(vals1.sum()) # This will cause an error (TypeError)!!!

[ ]: vals2 = np.array([1, np.nan, 3, 4])
      print(vals2)

      print(vals2.sum())
```

Performing arithmetic with `NaN` results in `NaN`:

```
[ ]: 1 + np.nan
```

```
[ ]: 0 * np.nan
```

As a result, aggregate functions will result in NaN:

```
[ ]: vals2.sum(), vals2.min(), vals2.max()
```

However, NumPy does provide a NaN-aware version of each aggregation function which ignore missing values:

```
[ ]: np.nansum(vals2), np.nanmin(vals2), np.nanmax(vals2)
```

1.3.2 Detecting null values

Pandas offers methods to detect, removing and replace null values:

```
[ ]: data = pd.Series([1, np.nan, 'hello', None])
```

```
[ ]: data.isnull()
```

We can use Boolean masks to remove null values from the DataFrame:

```
[ ]: data[data.notnull()]
```

Functions `isnull()` and `notnull()` produce Boolean results.

1.3.3 Dropping null values

To remove null values from the DataFrame, we can use `dropna()`. To fill null values with some constant, we can use `fillna()`:

```
[ ]: df = pd.DataFrame([[1,      np.nan, 2],  
                        [2,      3,     5],  
                        [np.nan, 4,     6]])  
  
df
```

```
[ ]: df.fillna(0)
```

```
[ ]: df.dropna()
```

```
[ ]: data = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'),  
                      dtype='Int32')  
  
data
```

```
[ ]: data.fillna(0)
```